UNIVERSIDAD CENFOTEC

Principios de Programación 1 (SOFT-01)

Profesor Facilitador:	Verónica Mora Lezcano
Curso:	Principios de Programación 1 (SOFT-01)
Estudiante:	Stwart Santamaria Artavia
Periodo Lectivo:	2026-C3
Tarea 2	Resolución de Problemas mediante Algoritmos y Programación en Python
Fecha de Entrega:	25 de octubre de 2026
Enlace al Repositorio Git:	https://github.com/stwartsantamaria-hub/C3_2026_Progra1/tree/8a7e29c689782edbbb90c5a4e9a50e5ba80550fa/Tarea_2

1. Contexto Seleccionado

El presente documento contiene el desarrollo completo de la solución al **Problema 1: Cajero Automático**. Se simula el proceso de retiros consecutivos sobre un saldo inicial fijo de ₡250 000, aplicando validaciones de límites por transacción (mínimo ₡5 000 y máximo ₡100 000), verificación de saldo disponible y solicitud explícita de continuidad al usuario mediante una estructura iterativa de tipo while.

2. Tablas de Datos de Entrada y Salida

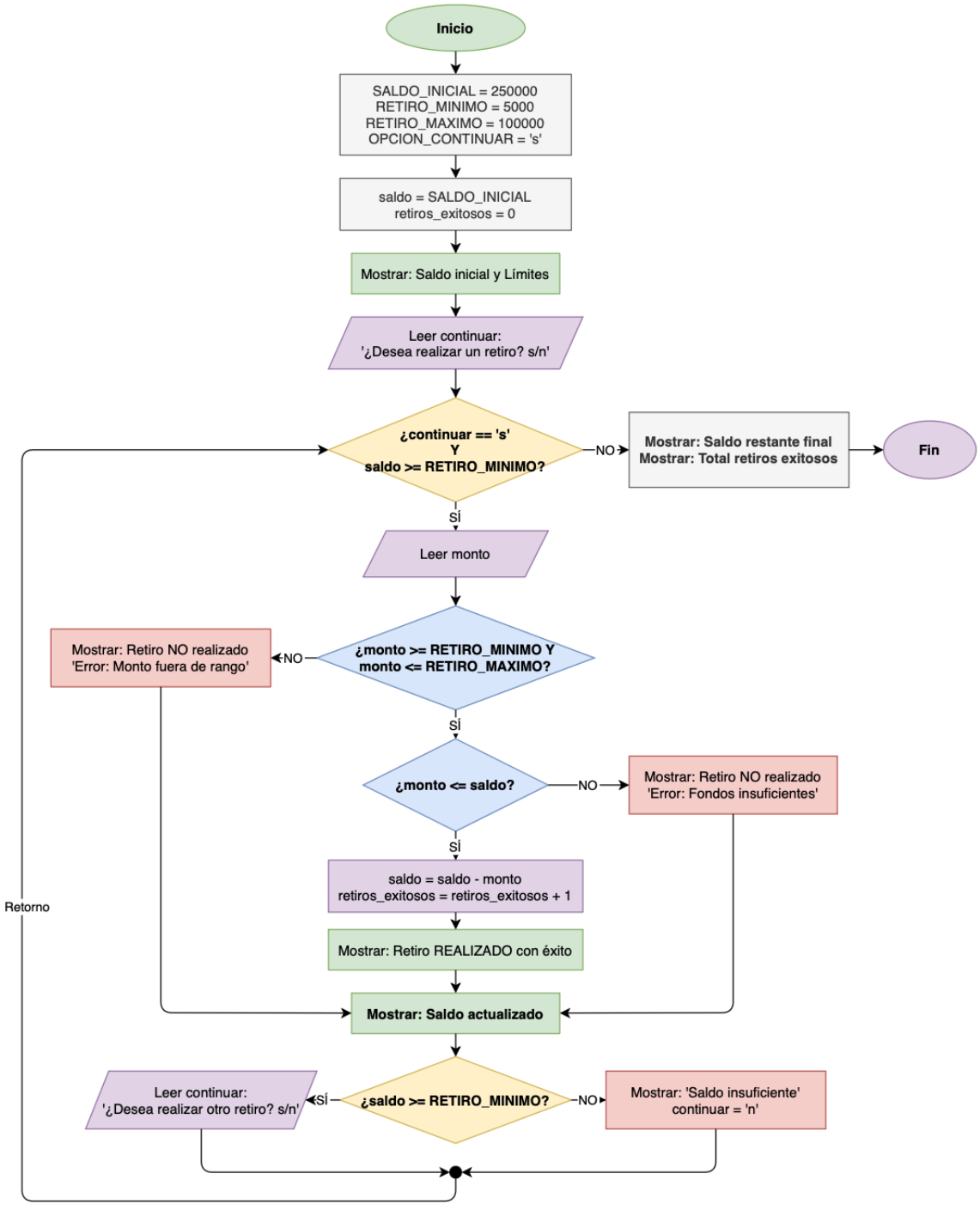
Tabla 2.1: Datos de Entrada

Nombre Variable / Constante	Tipo de Dato	Descripción	Valores Válidos
SALDO_INICIAL	Flotante (float)	Monto base de la cuenta bancaria.	Constante: 250000.0
RETIRO_MINIMO	Flotante (float)	Límite inferior permitido por retiro.	Constante: 5000.0
RETIRO_MAXIMO	Flotante (float)	Límite superior permitido por retiro.	Constante: 100000.0
OPCION_CONTINUAR	Cadena (str)	Opción afirmativa para evaluar el ciclo.	Constante: "s"
monto	Flotante (float)	Monto solicitado por el usuario en cada operación.	Números reales > 0
continuar	Cadena (str)	Confirmación digitada por el usuario para seguir operando.	"s", "S", "n", "N"

Tabla 2.2: Datos de Salida

Nombre Variable / Mensaje	Tipo de Dato	Descripción
Mensaje de Estado	Cadena (str)	Notifica si la transacción fue realizada con éxito o la causa específica del rechazo.
saldo	Flotante (float)	Muestra el saldo disponible actualizado tras cada intento de retiro y el saldo restante final.
retiros_exitosos	Entero (int)	Contador acumulativo que registra la cantidad total de transacciones completadas satisfactoriamente.

3. Diagrama de flujo



4. Código Fuente en Python

```python
# ============================================================================
# Nombre del estudiante: Stwart Santamaria Artavia
# Contexto seleccionado: Problema 1. Cajero automático
# Descripción: Simula retiros consecutivos en un cajero automático sobre un
#              saldo inicial de ₡250 000 utilizando un ciclo while.
# ============================================================================

# ----------------------------------------------------------------------------
# 1. CONSTANTES
# ----------------------------------------------------------------------------
SALDO_INICIAL = 250000.0   # Saldo inicial fijado por la regla de negocio
RETIRO_MINIMO = 5000.0     # Retiro mínimo por operación
RETIRO_MAXIMO = 100000.0   # Retiro máximo por operación
OPCION_CONTINUAR = "s"     # Opción afirmativa para continuar

# ----------------------------------------------------------------------------
# 2. INICIALIZACIÓN DE VARIABLES Y MUESTREO INICIAL
# ----------------------------------------------------------------------------
saldo = SALDO_INICIAL
retiros_exitosos = 0
monto = 0.0

# Mensaje de bienvenida con declaración explícita de límites
print("=========================================")
print("       BIENVENIDO AL CAJERO AUTOMÁTICO    ")
print("=========================================")
print("Saldo disponible inicial : ₡", saldo)
print(f"Límites por transacción  : Mínimo ₡{RETIRO_MINIMO:,.0f} | Máximo
  ₡{RETIRO_MAXIMO:,.0f}")
print("-----------------------------------------")

# Pregunta inicial para evaluar el ingreso al ciclo (Permite 0 iteraciones)
continuar = input("¿Desea realizar una operación de retiro? (s/n): ")

# ----------------------------------------------------------------------------
# 3. CICLO ITERATIVO WHILE
# ----------------------------------------------------------------------------
while continuar.lower() == OPCION_CONTINUAR and saldo >= RETIRO_MINIMO:

    # Lectura del monto recordando los límites al usuario
    monto = float(input(f"\nIngrese el monto a retirar: (Mínimo ₡{RETIRO_MINIMO:,.0f}
  | Máximo ₡{RETIRO_MAXIMO:,.0f}): ₡"))

    # Rombo 1: Validación de Límites (₡5 000 a ₡100 000)
    if monto >= RETIRO_MINIMO and monto <= RETIRO_MAXIMO:

        # Rombo 2: Validación de Fondos Disponibles
        if monto <= saldo:
            # Procesos cuando el retiro es exitoso
            saldo = saldo - monto
            retiros_exitosos = retiros_exitosos + 1
            print("Estado: Retiro REALIZADO con éxito.")
        else:
```

```
            # Mensaje cuando el monto supera el saldo disponible
            print("Estado: Retiro NO realizado (Error: Fondos insuficientes).")

    else:
        # Mensaje cuando el monto no cumple con el rango mínimo/máximo
        print(f"Estado: Retiro NO realizado (Error: El monto debe estar entre
 ₡{RETIRO_MINIMO:,.0f} y ₡{RETIRO_MAXIMO:,.0f}).")

    # Muestra del saldo actualizado por cada intento
    print("Saldo actualizado: ₡", saldo)

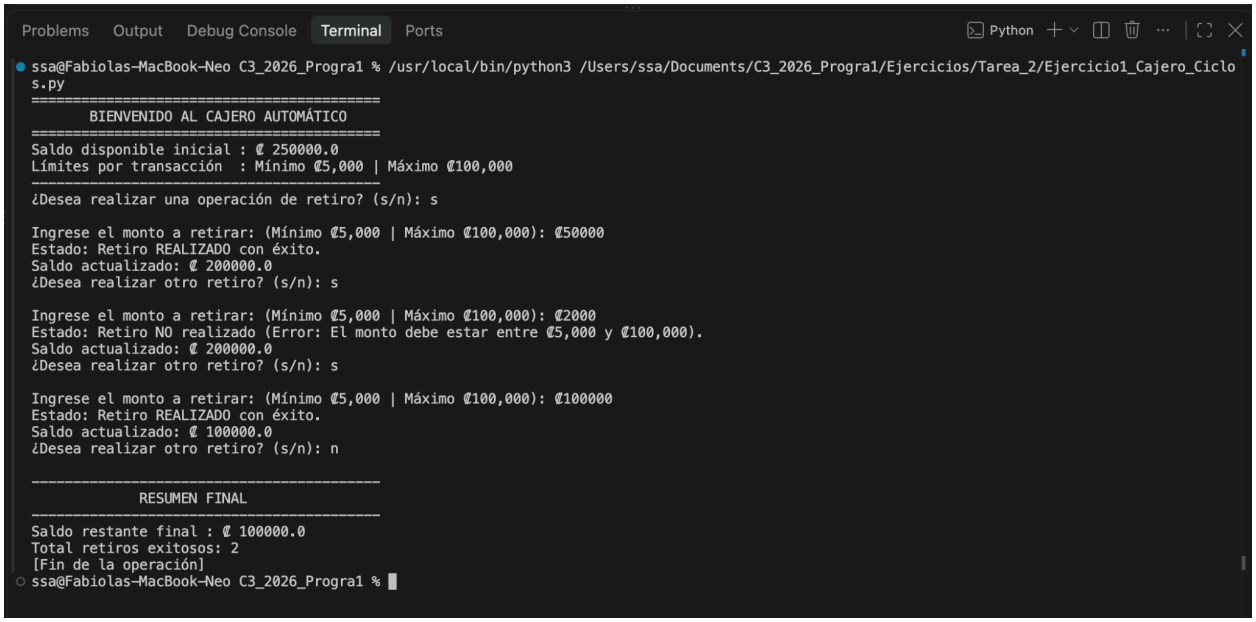
    # Evaluación para solicitar la confirmación de continuar
    if saldo >= RETIRO_MINIMO:
        continuar = input("¿Desea realizar otro retiro? (s/n): ")
    else:
        print(f"\nSaldo insuficiente para realizar más retiros (mínimo
 ₡{RETIRO_MINIMO:,.0f}).")
        continuar = "n"

# ------------------------------------------------------------------------------
# 4. SALIDA FINAL DE RESULTADOS
# ------------------------------------------------------------------------------
print("\n-----------------------------------------")
print("             RESUMEN FINAL               ")
print("-----------------------------------------")
print("Saldo restante final : ₡", saldo)
print("Total retiros exitosos:", retiros_exitosos)
print("[Fin de la operación]")
```

4. Pruebas de Escritorio (Trazas del Algoritmo)

Caso de Prueba 1: Caso Normal (Múltiples iteraciones con éxito y error)

Explicación: Simula el uso cotidiano del cajero. El usuario realiza varias operaciones consecutivas combinando retiros exitosos (ej. ₡50 000 y ₡100 000) y un intento fallido al ingresar un monto fuera de rango (ej. ₡2 000). Sirve para demostrar que el programa valida correctamente los límites y actualiza el saldo y contador progresivamente.



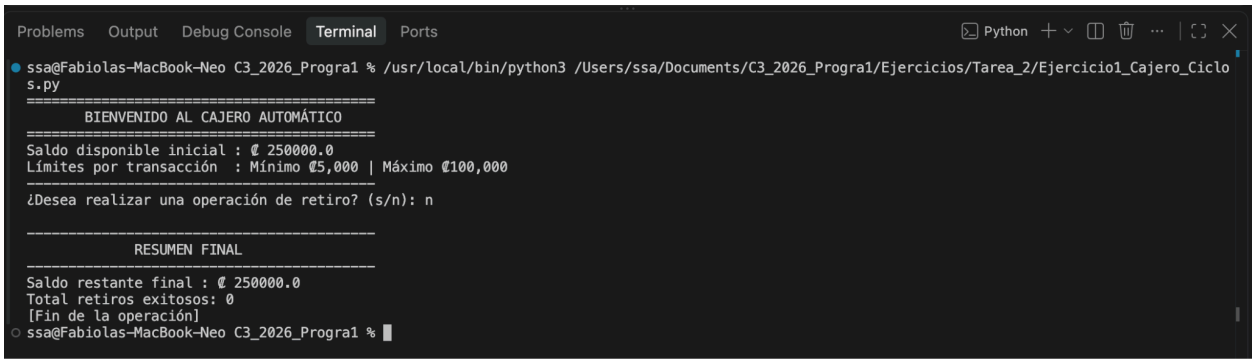
Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
0 (inicial)	continuar = "" saldo = 250000.0	N/A	N/A	Imprime bienvenida, saldo inicial (₡250 000.0) y límites (₡5 000 - ₡100 000). Se lee continuar =	saldo = 250000.0 retiros_exitosos = 0	BIENVENIDO AL CAJERO AUTOMÁTICO Saldo disponible inicial : ₡ 250000.0 Límites por

Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
				"s"		transacción : Mínimo ₡5,000 | Máximo ₡100,000 ¿Desea realizar una operación de retiro? (s/n): s
1	continuar = "s" saldo = 250000.0	"s" == "s" Y 250000.0 >= 5000.0	**Sí** (Verdadero)	Se lee monto = 50000.0 Evalúa 5000 <= 50000 <= 100000 (V) Evalúa 50000 <= 250000 (V) saldo = 250000 - 50000 retiros_exitos os = 0 + 1 Se lee continuar = "s"	saldo = 200000.0 retiros_exitos os = 1	Ingrese el monto a retirar (₡5,000 - ₡100,000): ₡ 50000 Estado: Retiro REALIZADO con éxito. Saldo actualizado: ₡ 200000.0 ¿Desea realizar otro retiro? (s/n): s
2	continuar = "s" saldo = 200000.0	"s" == "s" Y 200000.0 >= 5000.0	**Sí** (Verdadero)	Se lee monto = 2000.0 Evalúa 5000 <= 2000 <= 100000 (F) Se lee continuar = "s"	saldo = 200000.0 retiros_exitos os = 1	Ingrese el monto a retirar (₡5,000 - ₡100,000): ₡ 2000 Estado: Retiro NO realizado (Error: El monto debe

Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
						estar entre...) Saldo actualizado: ₡ 200000.0 ¿Desea realizar otro retiro? (s/n): s
3	continuar = "s" saldo = 200000.0	"s" == "s" Y 200000.0 >= 5000.0	**Sí** (Verdadero)	Se lee monto = 100000.0 Evalúa 5000 <= 100000 <= 100000 (V) Evalúa 100000 <= 200000 (V) saldo = 200000 - 100000 retiros_exitosos = 1 + 1 Se lee continuar = "n"	saldo = 100000.0 retiros_exitosos = 2	Ingrese el monto a retirar (₡5,000 - ₡100,000): ₡ 100000 Estado: Retiro REALIZADO con éxito. Saldo actualizado: ₡ 100000.0 ¿Desea realizar otro retiro? (s/n): n
Final	continuar = "n" saldo = 100000.0	"n" == "s" Y 100000.0 >= 5000.0	**No** (Sale)	No se ejecuta el cuerpo.	saldo = 100000.0 retiros_exitosos = 2	RESUMEN FINAL Saldo restante final: ₡ 100000.0 Total retiros exitosos: 2

Caso de Prueba 2: Caso Cero Iteraciones (0 ejecuciones)

Explicación: Prueba la condición de entrada inicial. Al iniciar el programa y mostrar el saldo, si el usuario responde "n" a la pregunta *«¿Desea realizar una operación de retiro?»*, la condición del `while` es falsa desde el arranque. El programa salta el ciclo y muestra el resumen intacto (₡250 000 y 0 retiros exitosos).



Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
0 (inicial)	continuar = "" saldo = 250000.0	N/A	N/A	Imprime bienvenida, saldo inicial (₡250 000.0) y límites. Se lee continuar = "n"	saldo = 250000.0 retiros_exitosos = 0	BIENVENIDO AL CAJERO AUTOMÁTICO Saldo disponible inicial : ₡ 250000.0 Límites por transacción : Mínimo ₡5,000 | Máximo ₡100,000 ¿Desea realizar una operación de retiro? (s/n):

Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
						n
Final	continuar = "n" saldo = 250000.0	"n" == "s" Y 250000.0 >= 5000.0	**No** (Sale)	No se ejecuta el cuerpo del ciclo.	saldo = 250000.0 retiros_exitosos = 0	RESUMEN FINAL Saldo restante final : ₡ 250000.0 Total retiros exitosos: 0

Caso de Prueba 3: Caso Una Sola Iteración (1 ejecución)

Explicación: Evalúa la ejecución de una única transacción. El usuario indica que "s" desea operar, realiza un retiro válido (ej. ₡25 000), el sistema procesa el cambio de saldo a ₡225 000 y, al preguntarle si desea hacer otro retiro, responde "n" para salir de inmediato.



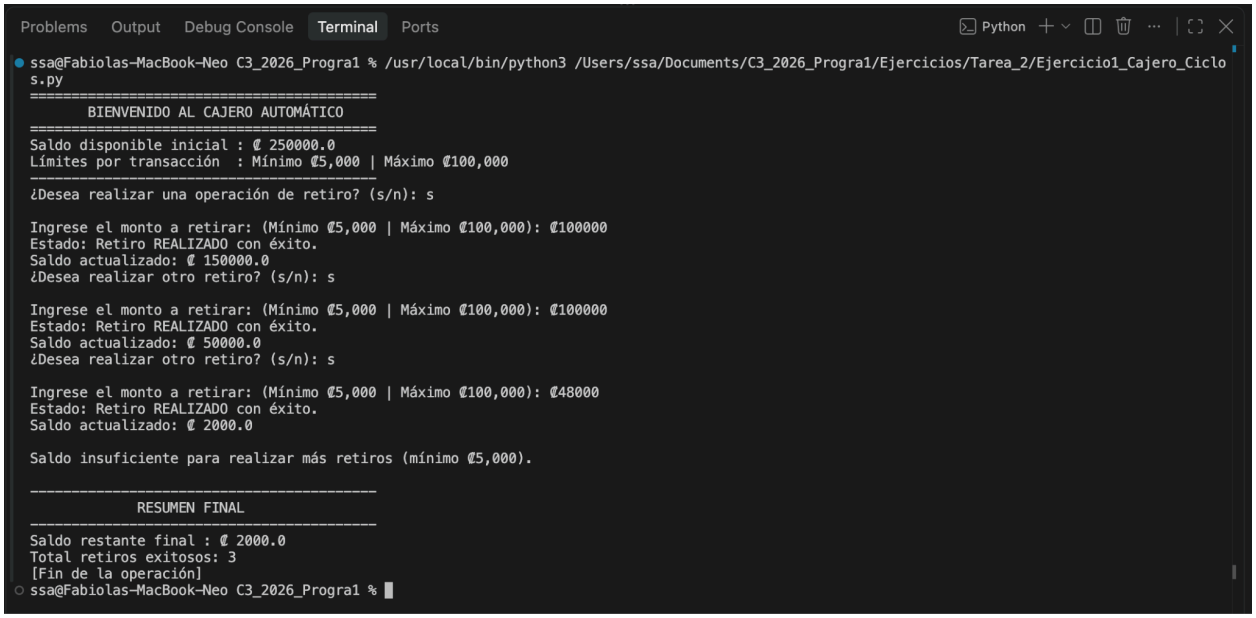
Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
0 (inicial)	continuar = "" saldo = 250000.0	N/A	N/A	Imprime bienvenida, saldo inicial (₡250 000.0)	saldo = 250000.0 retiros_exitos	BIENVENIDO AL CAJERO AUTOMÁTIC

Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
				y límites. Se lee continuar = "s"	os = 0	O Saldo disponible inicial : ₡ 250000.0 Límites por transacción : Mínimo ₡5,000 | Máximo ₡100,000 ¿Desea realizar una operación de retiro? (s/n): s
1	continuar = "s" saldo = 250000.0	"s" == "s" Y 250000.0 >= 5000.0	**Sí** (Verdadero)	Se lee monto = 25000.0 Evalúa 5000 <= 25000 <= 100000 (V) Evalúa 25000 <= 250000 (V) saldo = 250000 - 25000 retiros_exitos os = 0 + 1 Se lee continuar = "n"	saldo = 225000.0 retiros_exitos os = 1	Ingrese el monto a retirar (₡5,000 - ₡100,000): ₡ 25000 Estado: Retiro REALIZADO con éxito. Saldo actualizado: ₡ 225000.0 ¿Desea realizar otro retiro? (s/n): n
Final	continuar = "n" saldo = 225000.0	"n" == "s" Y 225000.0 >= 5000.0	**No** (Sale)	No se ejecuta el cuerpo del ciclo.	saldo = 225000.0 retiros_exitos os = 1	RESUMEN FINAL Saldo restante final : ₡ 225000.0

Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
						Total retiros exitosos: 1

Caso de Prueba 4: Caso Límite (Agotamiento de saldo disponible)

Explicación: Prueba la regla de negocio frente a fondos insuficientes para continuar. El usuario realiza varios retiros hasta reducir el saldo a un monto inferior al mínimo permitido (ej. quedan ₡2 000). El sistema detecta que `saldo < ₡5 000`, notifica que no es posible hacer más retiros y asigna automáticamente `continuar = "n"` para forzar la salida del ciclo.



Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
0 (inicial)	continuar = "" saldo = 250000.0	N/A	N/A	Imprime bienvenida, saldo inicial (₡250 000.0) y límites. Se lee	saldo = 250000.0 retiros_exitosos = 0	BIENVENIDO AL CAJERO AUTOMÁTICO Saldo

Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
				continuar = "s"		disponible inicial : ₡ 250000.0 Límites por transacción : Mínimo ₡5,000 | Máximo ₡100,000 ¿Desea realizar una operación de retiro? (s/n): s
1	continuar = "s" saldo = 250000.0	"s" == "s" Y 250000.0 >= 5000.0	**Sí** (Verdadero)	Se lee monto = 100000.0 Evalúa 5000 <= 100000 <= 100000 (V) Evalúa 100000 <= 250000 (V) saldo = 250000 - 100000 retiros_exitosos = 0 + 1 Se lee continuar = "s"	saldo = 150000.0 retiros_exitosos = 1	Ingrese el monto a retirar (₡5,000 - ₡100,000): ₡ 100000 Estado: Retiro REALIZADO con éxito. Saldo actualizado: ₡ 150000.0 ¿Desea realizar otro retiro? (s/n): s
2	continuar = "s" saldo = 150000.0	"s" == "s" Y 150000.0 >= 5000.0	**Sí** (Verdadero)	Se lee monto = 100000.0 Evalúa 5000 <= 100000 <= 100000 (V) Evalúa	saldo = 50000.0 retiros_exitosos = 2	Ingrese el monto a retirar (₡5,000 - ₡100,000): ₡ 100000 Estado:

Iteración	Variables de control (antes)	Condición evaluada	¿Entra al ciclo?	Cálculos / actualizaciones	Acumuladores (después)	Salida mostrada
				100000 <= 150000 (V) saldo = 150000 - 100000 retiros_exitosos = 1 + 1 Se lee continuar = "s"		Retiro REALIZADO con éxito. Saldo actualizado: ₡ 50000.0 ¿Desea realizar otro retiro? (s/n): s
3	continuar = "s" saldo = 50000.0	"s" == "s" Y 50000.0 >= 5000.0	**Sí** (Verdadero)	Se lee monto = 48000.0 Evalúa 5000 <= 48000 <= 100000 (V) Evalúa 48000 <= 50000 (V) saldo = 50000 - 48000 (2000.0) retiros_exitosos = 2 + 1 Evalúa saldo (2000.0) >= 5000.0 (F) Asigna continuar = "n"	saldo = 2000.0 retiros_exitosos = 3	Ingrese el monto a retirar (₡5,000 - ₡100,000): ₡ 48000 Estado: Retiro REALIZADO con éxito. Saldo actualizado: ₡ 2000.0 Saldo insuficiente para realizar más retiros (mínimo ₡5 000).
Final	continuar = "n" saldo = 2000.0	"n" == "s" Y 2000.0 >= 5000.0	**No** (Sale)	No se ejecuta el cuerpo del ciclo.	saldo = 2000.0 retiros_exitosos = 3	RESUMEN FINAL Saldo restante final : ₡ 2000.0 Total retiros exitosos: 3